

QLS Project — Complete Code Bundle

Every file's full source as built across past chats. Drop this into a new conversation along with the handoff doc and I can reconstruct the entire codebase.

Project root: `C:\Users\thotl\OneDrive\Desktop\QLS` (Windows venv, Python 3.10.7)

Directory structure

```
QLS/
├─ conftest.py           (empty marker)
├─ setup.py
├─ venv/                 (Python 3.10 virtualenv)
├─ src/
│   ├─ __init__.py      (empty)
│   ├─ gain_cache.py
│   ├─ local_search.py
│   ├─ qubo.py
│   ├─ graph.py
│   ├─ backends.py
│   ├─ selectors.py
│   ├─ qls.py
│   ├─ adaptive_qls.py
│   ├─ baselines.py
│   ├─ metrics.py
│   └─ experiment.py
├─ tests/
│   ├─ __init__.py      (empty)
│   ├─ test_qubo.py
│   └─ test_local_search.py
├─ data/
│   ├─ gset/             (G1.txt, G11.txt, G14.txt, G22.txt)
│   └─ generated/
├─ results/              (JSON output)
└─ notebooks/
```

Setup

setup.py

```
python
```

```
from setuptools import setup, find_packages

setup(
    name="qls_maxcut",
    version="0.1",
    packages=find_packages(),
)
```

conftest.py — empty file (just a marker for pytest to find project root)

Install

```
bash

py -3.10 -m venv venv
venv\Scripts\activate
python -m pip install --upgrade pip
pip install -e .
pip install networkx numpy scipy scikit-learn dimod dwave-neal openjij pytest
```

src/gain_cache.py

```
python
```

```

"""
GainCache – explicit gain array management with invalidation.
Prevents stale gain bugs by hard-crashing if gains are read after
a solution change without recomputation.
"""

class GainCache:
    """
    Maintains flip-gain values for all vertices.
    gain[v] = improvement in cut value if vertex v is flipped.
    Positive gain means flipping v improves the cut.
    """

    def __init__(self):
        self.gain = {}
        self.valid = False

    def update(self, G, x):
        """Full recompute –  $O(|E|)$ . Call when solution changes significantly."""
        self.gain = {}
        for v in G.nodes():
            self.gain[v] = self._compute_gain(G, x, v)
        self.valid = True

    def incremental_update(self, G, x, flipped_v):
        """Partial recompute after one flip –  $O(\deg(v))$ ."""
        assert self.valid, "Cannot incrementally update invalid cache"
        self.gain[flipped_v] = self._compute_gain(G, x, flipped_v)
        for u in G.neighbors(flipped_v):
            self.gain[u] = self._compute_gain(G, x, u)

    def invalidate(self):
        """Call after any accepted QLS proposal – forces full recompute next use."""
        self.valid = False

    def assert_valid(self):
        """Call at the start of every selector – hard crash if stale."""
        assert self.valid, (
            "GainCache is stale. Call update() before reading gains."
        )

    def best_flip(self):
        """Returns (vertex, gain) for the best improving flip."""
        self.assert_valid()
        best_v = max(self.gain, key=lambda v: self.gain[v])

```

```
        return best_v, self.gain[best_v]

def _compute_gain(self, G, x, v):
    """
    Flip gain for vertex v given current assignment x.
    gain > 0 means flipping v increases the cut.
    """
    gain = 0
    for u in G.neighbors(v):
        w = G[v][u].get('weight', 1.0)
        if x[v] == x[u]:
            gain += w    # currently NOT cut – flipping v would cut this edge
        else:
            gain -= w    # currently cut – flipping v would uncut this edge
    return gain
```

src/local_search.py

python

```
"""
```

```
OneFlipLS – best-improvement one-flip local search for Max-Cut.  
Returns the solution, updated GainCache, and a flag indicating  
whether a one-flip local optimum was reached.
```

```
"""
```

```
from src.gain_cache import GainCache
```

```
def compute_cut_value(G, x):
```

```
    """Total weight of edges crossing the cut."""
```

```
    cut = 0.0
```

```
    for u, v, data in G.edges(data=True):
```

```
        w = data.get('weight', 1.0)
```

```
        if x[u] != x[v]:
```

```
            cut += w
```

```
    return cut
```

```
def one_flip_ls(G, x, gc, max_iter=None):
```

```
    """
```

```
    Best-improvement one-flip local search.
```

```
    Parameters
```

```
    -----
```

```
    G          : NetworkX graph
```

```
    x          : dict {vertex: 0 or 1} – current assignment
```

```
    gc         : GainCache instance
```

```
    max_iter   : int or None – iteration budget (None = run to local optimum)
```

```
    Returns
```

```
    -----
```

```
    x          : improved assignment
```

```
    gc         : updated GainCache
```

```
    at_opt     : True if stopped at one-flip local optimum
```

```
    n_flips    : number of flips performed
```

```
    """
```

```
    x = dict(x)          # work on a copy
```

```
    gc.update(G, x)      # always recompute on entry
```

```
    n_flips = 0
```

```
    iter_count = 0
```

```
    while True:
```

```
        if max_iter is not None and iter_count >= max_iter:
```

```
            return x, gc, False, n_flips
```

```
best_v, best_gain = gc.best_flip()

if best_gain <= 0:
    # no improving flip exists – at local optimum
    return x, gc, True, n_flips

# apply the best flip
x[best_v] = 1 - x[best_v]
gc.incremental_update(G, x, best_v)

n_flips += 1
iter_count += 1

def random_cut(G, rng=None):
    """
    Generate a random binary assignment for all vertices.
    """
    import numpy as np
    if rng is None:
        rng = np.random.default_rng()
    return {v: int(rng.integers(0, 2)) for v in G.nodes()}
```

src/qubo.py

python

```
"""
```

```
QUBO subproblem construction for Max-Cut local search.  
BuildLocalQUBO extracts a local QUBO model over a selected  
vertex subset S, with boundary conditions fixed by the incumbent.
```

```
"""
```

```
def build_local_qubo(G, x, S):
```

```
    """
```

```
    Build a QUBO dict for the subgraph induced by vertex subset S.  
    Vertices outside S are fixed to their current assignment in x.
```

```
    Minimising the returned Q is equivalent to maximising the  
    cut contribution of S given the fixed boundary.
```

```
    """
```

```
    S = set(S)
```

```
    Q = {}
```

```
    for u, v, data in G.edges(data=True):
```

```
        w = data.get('weight', 1.0)
```

```
        u_in = u in S
```

```
        v_in = v in S
```

```
        if u_in and v_in:
```

```
            # internal edge – both endpoints free to optimise
```

```
            Q[(u, u)] = Q.get((u, u), 0.0) - w
```

```
            Q[(v, v)] = Q.get((v, v), 0.0) - w
```

```
            Q[(u, v)] = Q.get((u, v), 0.0) + 2.0 * w
```

```
        elif u_in and not v_in:
```

```
            # boundary edge – v is fixed at x[v]
```

```
            if x[v] == 0:
```

```
                Q[(u, u)] = Q.get((u, u), 0.0) - w
```

```
            else:
```

```
                Q[(u, u)] = Q.get((u, u), 0.0) + w
```

```
        elif v_in and not u_in:
```

```
            # boundary edge – u is fixed at x[u]
```

```
            if x[u] == 0:
```

```
                Q[(v, v)] = Q.get((v, v), 0.0) - w
```

```
            else:
```

```
                Q[(v, v)] = Q.get((v, v), 0.0) + w
```

```
    return Q
```

```
def qubo_energy(Q, x_local, S):
    """
    Evaluate QUBO energy for a given local assignment.
    Lower energy = better cut within S.
    """
    S = list(S)
    energy = 0.0
    for i in range(len(S)):
        vi = S[i]
        xi = x_local.get(vi, 0)
        energy += Q.get((vi, vi), 0.0) * xi
        for j in range(i + 1, len(S)):
            vj = S[j]
            xj = x_local.get(vj, 0)
            energy += Q.get((vi, vj), 0.0) * xi * xj
    return energy

def merge_proposal(x, x_local, S):
    """
    Merge a local assignment back into the full solution.
    """
    x_prop = dict(x)
    for v in S:
        if v in x_local:
            x_prop[v] = x_local[v]
    return x_prop
```

src/graph.py

```
python
```

```
"""
```

```
Graph layer – loading G-set instances and generating benchmark graphs.  
All graphs are NetworkX weighted graphs.
```

```
"""
```

```
import os  
import networkx as nx  
import numpy as np
```

```
def load_gset(filepath):  
    """  
    Load a G-set Max-Cut instance from file.  
    G-set format:  
        First line: n_vertices n_edges  
        Each subsequent line: u v weight  
    """  
    G = nx.Graph()  
  
    with open(filepath, 'r') as f:  
        lines = f.readlines()  
  
    first_line = lines[0].strip().split()  
    n_vertices = int(first_line[0])  
    n_edges = int(first_line[1])  
  
    for v in range(1, n_vertices + 1):  
        G.add_node(v)  
  
    for line in lines[1:]:  
        parts = line.strip().split()  
        if len(parts) < 2:  
            continue  
        u = int(parts[0])  
        v = int(parts[1])  
        w = float(parts[2]) if len(parts) > 2 else 1.0  
        G.add_edge(u, v, weight=w)  
  
    assert G.number_of_nodes() == n_vertices  
    assert G.number_of_edges() == n_edges  
  
    return G
```

```
def generate_random_regular(n, d, seed=None):  
    """Random d-regular unweighted graph on n vertices."""
```

```

G = nx.random_regular_graph(d, n, seed=seed)
for u, v in G.edges():
    G[u][v]['weight'] = 1.0
return G

def generate_erdos_renyi(n, p, seed=None):
    """Erdos-Renyi random graph G(n, p)."""
    G = nx.erdos_renyi_graph(n, p, seed=seed)
    for u, v in G.edges():
        G[u][v]['weight'] = 1.0
    return G

def generate_sbm(sizes, p_in, p_out, seed=None):
    """Stochastic Block Model graph with strong community structure when p_in >> p_out"""
    probs = []
    n_blocks = len(sizes)
    for i in range(n_blocks):
        row = []
        for j in range(n_blocks):
            row.append(p_in if i == j else p_out)
        probs.append(row)

    G = nx.stochastic_block_model(sizes, probs, seed=seed)
    for u, v in G.edges():
        G[u][v]['weight'] = 1.0
    return G

def graph_stats(G):
    """Print basic graph statistics."""
    n = G.number_of_nodes()
    m = G.number_of_edges()
    density = nx.density(G)
    degrees = [d for _, d in G.degree()]
    avg_degree = sum(degrees) / n if n > 0 else 0

    print(f"Vertices : {n}")
    print(f"Edges      : {m}")
    print(f"Density    : {density:.4f}")
    print(f"Avg deg    : {avg_degree:.2f}")
    print(f"Min deg    : {min(degrees)}")
    print(f"Max deg    : {max(degrees)}")

```

python

```
"""
```

```
Backend solvers for local QUBO subproblems.
```

```
All backends share the same interface:
```

```
    backend(Q, S, n_reads, init=None) -> x_local (dict {vertex: 0 or 1})
```

```
"""
```

```
import numpy as np
```

```
import dimod
```

```
from dwave.samplers import SimulatedAnnealingSampler as _NealSampler
```

```
def backend_exact(Q, S, n_reads=None, init=None):
```

```
    """Exact brute-force solver. Only use for |S| <= 20."""
```

```
    S = list(S)
```

```
    n = len(S)
```

```
    assert n <= 20, f"Exact solver only for |S| <= 20, got |S|={n}"
```

```
    best_energy = float('inf')
```

```
    best_x = None
```

```
    for mask in range(2 ** n):
```

```
        x_local = {}
```

```
        for i, v in enumerate(S):
```

```
            x_local[v] = (mask >> i) & 1
```

```
        energy = 0.0
```

```
        for i in range(n):
```

```
            vi = S[i]
```

```
            xi = x_local[vi]
```

```
            energy += Q.get((vi, vi), 0.0) * xi
```

```
            for j in range(i + 1, n):
```

```
                vj = S[j]
```

```
                xj = x_local[vj]
```

```
                energy += Q.get((vi, vj), 0.0) * xi * xj
```

```
        if energy < best_energy:
```

```
            best_energy = energy
```

```
            best_x = dict(x_local)
```

```
    return best_x
```

```
def backend_neal(Q, S, n_reads=100, init=None):
```

```
    """
```

```
    Simulated annealing via D-Wave neal (now in dwave.samplers).
```

```
    Quantum-inspired classical baseline.
```

```

"""
S = list(S)

# build dimod BQM from our QUBO dict
bqm = dimod.BinaryQuadraticModel('BINARY')
for (i, j), val in Q.items():
    if i == j:
        bqm.add_variable(i, val)
    else:
        bqm.add_interaction(i, j, val)

sampler = _NealSampler()

# prepare warm start if provided
initial_states = None
if init is not None:
    initial_states = [{v: init[v] for v in S if v in init}]

response = sampler.sample(
    bqm,
    num_reads=n_reads,
    initial_states=initial_states
)

best_sample = response.first.sample
return {v: best_sample[v] for v in S if v in best_sample}

def backend_sqa(Q, S, n_reads=100, init=None):
    """
    Simulated Quantum Annealing via OpenJij.
    Quantum-inspired with transverse field simulation.
    """
    try:
        import openjij as oj
    except ImportError:
        raise ImportError("OpenJij not installed. Run: pip install openjij")

    S = list(S)

    Q_oj = {}
    for (i, j), val in Q.items():
        Q_oj[(i, j)] = val

    sampler = oj.SQASampler()

    response = sampler.sample_qubo(

```

```
        Q_oj,
        num_reads=n_reads
    )

    best_sample = response.first.sample
    return {v: best_sample.get(v, 0) for v in S}


def get_backend(name):
    """Factory function – returns backend by name."""
    backends = {
        'exact': backend_exact,
        'neal':  backend_neal,
        'sqa':   backend_sqa,
    }
    if name not in backends:
        raise ValueError(
            f"Unknown backend '{name}'. Choose from: {list(backends.keys())}"
        )
    return backends[name]
```

src/selectors.py

```
python
```

```
"""
```

Neighbourhood selectors for Adaptive QLS.

All selectors share the same interface:

```
selector(G, gc, k, pool=None, rng=None) -> S (list of vertices)
```

```
"""
```

```
import numpy as np
```

```
from src.gain_cache import GainCache
```

```
def _plateau_aware_rank(G, gc, mode):
```

```
    """
```

Rank vertices by gain with plateau detection.

If >80% of vertices have |gain| near zero (plateau),

use degree as a tiebreaker.

mode : 'frustrated' (ascending |gain|) or 'impact' (descending |gain|)

```
    """
```

```
    gc.assert_valid()
```

```
    nodes = list(G.nodes())
```

```
    eps = 1e-6
```

```
    frac_zero = sum(1 for v in nodes if abs(gc.gain[v]) < eps) / len(nodes)
```

```
    if frac_zero > 0.8:
```

```
        # plateau detected – use degree as tiebreaker
```

```
        if mode == 'frustrated':
```

```
            ranked = sorted(nodes,
```

```
                            key=lambda v: (abs(gc.gain[v]), -G.degree(v)))
```

```
        else:
```

```
            ranked = sorted(nodes,
```

```
                            key=lambda v: (-abs(gc.gain[v]), -G.degree(v)))
```

```
    else:
```

```
        if mode == 'frustrated':
```

```
            ranked = sorted(nodes, key=lambda v: abs(gc.gain[v]))
```

```
        else:
```

```
            ranked = sorted(nodes, key=lambda v: -abs(gc.gain[v]))
```

```
    return ranked
```

```
def select_random(G, gc, k, pool=None, rng=None):
```

```
    """Random selector – Tomesh et al. 2022 baseline."""
```

```
    gc.assert_valid()
```

```
    if rng is None:
```

```
        rng = np.random.default_rng()
```

```

nodes = list(G.nodes())
k_actual = min(k, len(nodes))
return list(rng.choice(nodes, size=k_actual, replace=False))

def select_frustrated(G, gc, k, pool=None, rng=None):
    """Frustrated – vertices with |gain| closest to zero. Novel."""
    gc.assert_valid()
    ranked = _plateau_aware_rank(G, gc, mode='frustrated')
    return ranked[:min(k, len(ranked))]

def select_frustrated_connected(G, gc, k, pool=None, rng=None):
    """
    Connected frustrated selector – novel contribution.
    Seeds from the most frustrated vertex then grows a connected
    subgraph by adding neighbours with lowest |gain|.
    """
    gc.assert_valid()
    nodes = list(G.nodes())
    eps = 1e-6

    if len(nodes) <= k:
        return nodes

    frac_zero = sum(1 for v in nodes if abs(gc.gain[v]) < eps) / len(nodes)

    if frac_zero > 0.8:
        seed = min(nodes, key=lambda v: (abs(gc.gain[v]), -G.degree(v)))
    else:
        seed = min(nodes, key=lambda v: abs(gc.gain[v]))

    # grow connected subgraph greedily
    S = [seed]
    frontier = set(G.neighbors(seed))

    while len(S) < k and frontier:
        best = min(frontier, key=lambda v: abs(gc.gain[v]))
        S.append(best)

        for u in G.neighbors(best):
            if u not in S:
                frontier.add(u)
        frontier.discard(best)

    # fill remaining slots with global frustrated vertices if frontier exhausted
    if len(S) < k:

```

```

        remaining = [v for v in nodes if v not in set(S)]
        remaining_sorted = sorted(remaining,
                                   key=lambda v: abs(gc.gain[v]))
        S.extend(remaining_sorted[:k - len(S)])

    return S[:k]

def select_impact(G, gc, k, pool=None, rng=None):
    """Impact – vertices with largest |gain|. qbsolv / Atobe 2022."""
    gc.assert_valid()
    ranked = _plateau_aware_rank(G, gc, mode='impact')
    return ranked[:min(k, len(ranked))]

def select_clustering(G, gc, k, pool=None, rng=None):
    """
    Clustering selector – correlation-based subgraph selection.
    Based on Zhao & Tang 2025.
    Falls back to select_frustrated if pool is too small.
    """
    gc.assert_valid()
    min_pool = 5

    if pool is None or len(pool) < min_pool:
        return select_frustrated(G, gc, k, pool=pool, rng=rng)

    nodes = list(G.nodes())
    n = len(nodes)

    pool_array = np.array(
        [[p[v] for v in nodes] for p in pool],
        dtype=float
    )

    # use cached clusters if available
    if (hasattr(select_clustering, '_cache') and
        select_clustering._cache.get('pool_size') == len(pool)):
        clusters = select_clustering._cache['clusters']
    else:
        try:
            from sklearn.cluster import SpectralClustering
            n_clusters = max(2, n // k)
            n_clusters = min(n_clusters, len(pool))

            C = np.corrcoef(pool_array.T)
            C = (C + 1) / 2 # normalise to [0, 1]

```

```

sc = SpectralClustering(
    n_clusters=n_clusters,
    affinity='precomputed',
    random_state=42,
    n_init=3
)
labels = sc.fit_predict(C)

clusters = {}
for i, v in enumerate(nodes):
    c = labels[i]
    if c not in clusters:
        clusters[c] = []
    clusters[c].append(v)

select_clustering._cache = {
    'pool_size': len(pool),
    'clusters': clusters
}

```

```

except Exception:

```

```

    return select_frustrated(G, gc, k, pool=pool, rng=rng)

```

```

# pick cluster with highest total |gain|

```

```

best_cluster_id = max(
    clusters.keys(),
    key=lambda c: sum(abs(gc.gain[v]) for v in clusters[c])
)
best_cluster = clusters[best_cluster_id]

```

```

if rng is None:

```

```

    rng = np.random.default_rng()

```

```

k_actual = min(k, len(best_cluster))

```

```

return list(rng.choice(best_cluster, size=k_actual, replace=False))

```

```

def select_meta_rule(G, gc, k, pool=None, rng=None):

```

```

    """

```

```

    Rule-based meta-selector.

```

```

    Switches between selectors based on current search state.

```

```

    Rules:

```

- plateau (>80% zero gain) -> impact + degree tiebreaker
- low pool diversity -> frustrated
- pool large enough -> clustering

```

- default                                -> frustrated
"""

gc.assert_valid()
nodes = list(G.nodes())
eps = 1e-6

frac_zero = sum(1 for v in nodes if abs(gc.gain[v]) < eps) / len(nodes)

# rule 1: plateau detected
if frac_zero > 0.8:
    return select_impact(G, gc, k, pool=pool, rng=rng)

# rule 2: pool diversity check
if pool is not None and len(pool) >= 2:
    pool_array = np.array(
        [[p[v] for v in nodes] for p in pool], dtype=float
    )
    n_pool = len(pool_array)
    diffs = 0
    count = 0
    for i in range(n_pool):
        for j in range(i + 1, n_pool):
            diffs += np.sum(pool_array[i] != pool_array[j])
            count += len(nodes)
    diversity = diffs / count if count > 0 else 1.0

    if diversity < 0.1:
        return select_frustrated(G, gc, k, pool=pool, rng=rng)

# rule 3: clustering if pool large enough
if pool is not None and len(pool) >= 5:
    return select_clustering(G, gc, k, pool=pool, rng=rng)

# default
return select_frustrated(G, gc, k, pool=pool, rng=rng)

def get_selector(name):
    """Factory function – returns selector by name."""
    selectors = {
        'random': select_random,
        'frustrated': select_frustrated,
        'frustrated_connected': select_frustrated_connected,
        'impact': select_impact,
        'clustering': select_clustering,
        'meta_rule': select_meta_rule,
    }

```

```
if name not in selectors:
    raise ValueError(
        f"Unknown selector '{name}'. "
        f"Choose from: {list(selectors.keys())}"
    )
return selectors[name]
```

src/metrics.py

python

```
"""
```

Metrics object – records all outcome and process metrics for a single algorithm run.

```
"""
```

```
import time
import numpy as np
```

```
class Metrics:
```

```
    """
```

Records everything needed for evaluation:

- Outcome metrics: cut value, approximation ratio, time-to-target
- Process metrics: escape rate, improvement per call, k trace

```
    """
```

```
def __init__(self, target_ratio=0.99):
    self.target_ratio = target_ratio
```

```
    # outcome metrics
```

```
    self.cut_trace = []
```

```
    self.best_cut = 0.0
```

```
    self.time_to_target = None
```

```
    # process metrics
```

```
    self.qls_calls = 0
```

```
    self.escape_successes = 0
```

```
    self.delta_trace = []
```

```
    self.k_trace = []
```

```
    self.paradigm_trace = []
```

```
    self.plateau_frac_trace = []
```

```
    self.selector_trace = []
```

```
    # timing
```

```
    self.start_time = None
```

```
    self.elapsed_trace = []
```

```
def start(self):
    self.start_time = time.time()
```

```
def record_cut(self, cut):
    self.cut_trace.append(cut)
    elapsed = time.time() - self.start_time
    self.elapsed_trace.append(elapsed)
    if cut > self.best_cut:
        self.best_cut = cut
```

```

def record_qls_call(self, delta, k, paradigm='unknown',
                    plateau_frac=0.0, selector='unknown'):
    self.qls_calls += 1
    self.delta_trace.append(delta)
    self.k_trace.append(k)
    self.paradigm_trace.append(paradigm)
    self.plateau_frac_trace.append(plateau_frac)
    self.selector_trace.append(selector)

    if delta > 0:
        self.escape_successes += 1

def check_time_to_target(self, cut, best_known):
    if self.time_to_target is None and best_known > 0:
        ratio = cut / best_known
        if ratio >= self.target_ratio:
            self.time_to_target = time.time() - self.start_time

def escape_rate(self):
    if self.qls_calls == 0:
        return 0.0
    return self.escape_successes / self.qls_calls

def mean_improvement(self):
    if not self.delta_trace:
        return 0.0
    return float(np.mean(self.delta_trace))

def summary(self, best_known=None):
    print(f"Best cut          : {self.best_cut:.1f}")
    if best_known:
        ratio = self.best_cut / best_known
        print(f"Approx ratio       : {ratio:.4f}")
    print(f"QLS calls          : {self.qls_calls}")
    print(f"Escape rate        : {self.escape_rate():.3f}")
    print(f"Mean delta         : {self.mean_improvement():.3f}")
    if self.time_to_target:
        print(f"Time-to-target     : {self.time_to_target:.2f}s")
    else:
        print(f"Time-to-target     : not reached")
    print(f"Total time         : {self.elapsed_trace[-1]:.2f}s"
          if self.elapsed_trace else "Total time: 0s")

```

python

```
"""
```

```
QLS – Quantum Local Search baseline.  
Random neighbourhood selector, fixed k, no adaptation.  
Based on Tomesh, Saleem and Suchara (2022) pattern.
```

```
"""
```

```
import time  
import numpy as np  
from src.gain_cache import GainCache  
from src.local_search import one_flip_ls, compute_cut_value, random_cut  
from src.qubo import build_local_qubo, merge_proposal  
from src.metrics import Metrics
```

```
def qls(G, budget_seconds, k, backend, n_reads=100,  
        best_known=None, seed=None):  
    """QLS baseline – random selector, fixed k, no adaptation."""  
    rng = np.random.default_rng(seed)  
    gc = GainCache()  
    metrics = Metrics()  
    metrics.start()  
  
    # initialise  
    x = random_cut(G, rng)  
    x_best = dict(x)  
    best_cut = compute_cut_value(G, x)  
  
    nodes = list(G.nodes())  
  
    while True:  
        if time.time() - metrics.start_time >= budget_seconds:  
            break  
  
        # Phase 1: classical descent to local optimum  
        x, gc, at_opt, n_flips = one_flip_ls(G, x, gc)  
        current_cut = compute_cut_value(G, x)  
        metrics.record_cut(current_cut)  
  
        if current_cut > best_cut:  
            best_cut = current_cut  
            x_best = dict(x)  
  
        if best_known:  
            metrics.check_time_to_target(current_cut, best_known)  
  
        if time.time() - metrics.start_time >= budget_seconds:
```

```
        break

# Phase 2: random neighbourhood selection
k_actual = min(k, len(nodes))
S = list(rng.choice(nodes, size=k_actual, replace=False))

# Phase 3: local solve – skip trivial subproblems
Q = build_local_qubo(G, x, S)
if not Q or all(abs(v) < 1e-10 for v in Q.values()):
    continue
x_local = backend(Q, S, n_reads=n_reads)

# Phase 4: merge and evaluate
x_prop = merge_proposal(x, x_local, S)
delta = compute_cut_value(G, x_prop) - current_cut

metrics.record_qls_call(
    delta=delta,
    k=k_actual,
    paradigm='qls',
    selector='random'
)

# Phase 5: accept improvement only
if delta > 0:
    x = x_prop
    gc.invalidate()
else:
    # restart on failure
    x = random_cut(G, rng)
    gc.invalidate()

return x_best, metrics
```

src/adaptive_qls.py

```
python
```

```
"""
```

```
Adaptive QLS – research contribution.  
Adaptive trigger (EMA-based), pluggable selector,  
momentum-damped k update, pool persistence on restart,  
look-ahead acceptance, SA cooling option.
```

```
"""
```

```
import time  
import numpy as np  
from src.gain_cache import GainCache  
from src.local_search import one_flip_ls, compute_cut_value, random_cut  
from src.qubo import build_local_qubo, merge_proposal  
from src.metrics import Metrics
```

```
def adaptive_qls(G, budget_seconds, selector, backend,  
                 k_min=5, k_max=30, n_reads=100,  
                 best_known=None, seed=None,  
                 acceptance='improvement',  
                 T_initial=2.0, T_min=0.001, cooling=0.995):
```

```
    """
```

```
    Adaptive QLS – pluggable selector, EMA trigger, adaptive k.
```

```
    acceptance options:
```

```
        'improvement' – accept only  $\Delta > 0$  (default)  
        'sa_cooling' – SA-style cooling schedule  
        'fixed_temp' – Liu & Goan style fixed-T Metropolis  
        'lookahead' – run one_flip_ls on x_prop before evaluating  $\Delta$ 
```

```
    """
```

```
    rng = np.random.default_rng(seed)  
    gc = GainCache()  
    metrics = Metrics()  
    metrics.start()
```

```
    nodes = list(G.nodes())
```

```
    # — pool warm-start: 5 diversified cuts —————
```

```
    pool = []  
    for i in range(5):  
        x_init = random_cut(G, rng)  
        gc_temp = GainCache()  
        x_opt, gc_temp, _, _ = one_flip_ls(G, x_init, gc_temp)  
        pool.append(dict(x_opt))
```

```
    # start from best pool solution
```

```
    x = max(pool, key=lambda p: compute_cut_value(G, p))
```

```

x = dict(x)
x_best = dict(x)
best_cut = compute_cut_value(G, x)

# — adaptive state —————
k = k_min
ema_esc = 0.5          # EMA escape probability
alpha = 0.3           # EMA smoothing factor
k_window = []         # gain window for k update
k_cooldown = 0        # prevent k oscillation
T = T_initial         # for sa_cooling acceptance

while time.time() - metrics.start_time < budget_seconds:

    # — Phase 1: classical descent —————
    x, gc, at_opt, n_flips = one_flip_ls(G, x, gc)
    current_cut = compute_cut_value(G, x)
    metrics.record_cut(current_cut)

    if current_cut > best_cut:
        best_cut = current_cut
        x_best = dict(x)
        if best_known:
            metrics.check_time_to_target(best_cut, best_known)

    if time.time() - metrics.start_time >= budget_seconds:
        break

    # — Phase 2: EMA-based adaptive trigger —————
    if not at_opt and ema_esc >= 0.2:
        continue

    # — Phase 3: plateau detection metric —————
    eps = 1e-6
    frac_zero = sum(
        1 for v in nodes if abs(gc.gain[v]) < eps
    ) / len(nodes)

    # — Phase 4: pluggable selector —————
    k_actual = min(k, len(nodes))
    try:
        S = selector(G, gc, k_actual, pool=pool, rng=rng)
    except Exception:
        S = list(rng.choice(nodes, size=k_actual, replace=False))

    # — Phase 5: local solve —————
    Q = build_local_qubo(G, x, S)

```

```

if not Q or all(abs(v) < 1e-10 for v in Q.values()):
    continue
x_local = backend(Q, S, n_reads=n_reads)
x_prop = merge_proposal(x, x_local, S)

# — Phase 5b: look-ahead acceptance —————
# run descent from proposed solution before deciding to accept
# this ensures we compare local optima to local optima
if acceptance == 'lookahead':
    gc_temp = GainCache()
    x_prop, gc_temp, _, _ = one_flip_ls(G, x_prop, gc_temp)

delta = compute_cut_value(G, x_prop) - current_cut

sel_name = getattr(selector, '__name__', 'unknown')
metrics.record_qls_call(
    delta=delta,
    k=k_actual,
    paradigm='adaptive_qls',
    plateau_frac=frac_zero,
    selector=sel_name
)

# — Phase 6: acceptance + pool update —————
accepted = False

if delta > 0:
    accepted = True
elif acceptance == 'sa_cooling':
    if T > T_min:
        prob = float(np.exp(delta / T))
        if rng.random() < prob:
            accepted = True
elif acceptance == 'fixed_temp':
    gamma = 1.0 / T_initial
    prob = float(np.exp(-gamma * abs(delta)))
    if rng.random() < prob:
        accepted = True
# else: improvement / lookahead – accepted stays False

if accepted:
    x = x_prop
    gc.invalidate()
    pool.append(dict(x))
    if len(pool) > 20:
        pool.pop(0)
else:

```

```

# pool persistence on restart – keep top-5
if ema_esc < 0.05:
    top5 = sorted(pool,
                   key=lambda p: compute_cut_value(G, p),
                   reverse=True)[:5]
    pool = list(top5)
    x = random_cut(G, rng)
    gc.invalidate()

# — Phase 6b: cool the temperature —————
if acceptance == 'sa_cooling':
    T = max(T_min, T * cooling)

# — Phase 7: EMA update —————
ema_esc = alpha * float(delta > 0) + (1 - alpha) * ema_esc

# — Phase 8: momentum-damped k update —————
k_window.append(delta)
k_cooldown -= 1

if len(k_window) >= 10 and k_cooldown <= 0:
    w = k_window[-9:]
    t0 = np.mean(w[0:3])
    t1 = np.mean(w[3:6])
    t2 = np.mean(w[6:9])

    if t0 < t1 < t2:
        # consistent improvement trend – shrink k
        k = max(k_min, k - 2)
        k_cooldown = 5
    elif t0 > t1 > t2:
        # consistent worsening trend – grow k
        k = min(k_max, k + 2)
        k_cooldown = 5

    k_window = [] # reset window after decision

return x_best, metrics

```

src/baselines.py

python

```
"""
```

```
Classical baselines for fair comparison against QLS.
```

- Improved Simulated Annealing (auto-calibrated T, linear-in-beta, gain-weighted selection, reheating)
- Tabu Search for Max-Cut

```
"""
```

```
import time
import numpy as np
from src.gain_cache import GainCache
from src.local_search import one_flip_ls, compute_cut_value, random_cut
from src.metrics import Metrics
```

```
# -----
# SA helper functions
# -----
```

```
def _calibrate_T_initial(G, x, gc, chi0=0.80, n_samples=200):
```

```
    """
```

```
    Ben-Ameur 2004 method – set T_initial so that chi0 fraction
    of worsening moves are accepted at the start.
```

```
    """
```

```
    nodes = list(G.nodes())
    rng = np.random.default_rng()
```

```
    bad_deltas = []
    for _ in range(n_samples):
        v = nodes[int(rng.integers(0, len(nodes)))]
        delta = gc.gain[v]
        if delta < 0:
            bad_deltas.append(abs(delta))
```

```
    if not bad_deltas:
        return 1.0
```

```
    # T such that mean acceptance of bad moves = chi0
    mean_bad = np.mean(bad_deltas)
    T = -mean_bad / np.log(chi0) if chi0 > 0 else mean_bad
    return max(T, 0.01)
```

```
def _gain_weighted_flip(G, x, gc, rng, T):
```

```
    """
```

```
    Gain-weighted vertex selection – 70% top-10% gain, 30% random.
```

```
    """
```

```

nodes = list(G.nodes())
n = len(nodes)

if rng.random() < 0.70:
    top_k = max(1, n // 10)
    candidates = sorted(nodes,
                        key=lambda v: gc.gain[v],
                        reverse=True)[:top_k]
    v = candidates[int(rng.integers(0, len(candidates)))]
else:
    v = nodes[int(rng.integers(0, n))]

delta = gc.gain[v]
return v, delta

```

```

# -----
# Improved Simulated Annealing
# -----

```

```

def simulated_annealing(G, budget_seconds, best_known=None, seed=None,
                        chi0=0.80, beta_final_multiplier=5.0,
                        reheat_threshold=10, max_reheats=3):
    """
    Improved simulated annealing for Max-Cut.

    Improvements over basic SA:
    1. Auto-calibrated T_initial (Ben-Ameur 2004)
    2. Linear-in-beta schedule across full budget
    3. Gain-weighted vertex selection
    4. Reheating when stagnated
    """
    rng = np.random.default_rng(seed)
    metrics = Metrics()
    metrics.start()

    nodes = list(G.nodes())
    n = len(nodes)

    x = random_cut(G, rng)
    gc = GainCache()
    gc.update(G, x)

    x_best = dict(x)
    best_cut = compute_cut_value(G, x)
    current_cut = best_cut

```

```

# — Step 1: calibrate T_initial —————
T_initial = _calibrate_T_initial(G, x, gc, chi0=chi0)
T = T_initial
beta = 1.0 / T

# — Step 2: set beta schedule —————
beta_initial = beta
beta_final = beta_initial * beta_final_multiplier

# reheating state
non_improving_flips = 0
reheat_count = 0
reheat_trigger = reheat_threshold * n

metrics.record_cut(current_cut)

while time.time() - metrics.start_time < budget_seconds:
    # linear-in-beta schedule
    elapsed_frac = (time.time() - metrics.start_time) / budget_seconds
    beta = beta_initial + (beta_final - beta_initial) * elapsed_frac
    T = 1.0 / beta if beta > 0 else T_initial

    # gain-weighted flip
    v, delta = _gain_weighted_flip(G, x, gc, rng, T)

    # Metropolis acceptance
    if delta > 0:
        x[v] = 1 - x[v]
        gc.incremental_update(G, x, v)
        current_cut += delta
        non_improving_flips = 0

        if current_cut > best_cut:
            best_cut = current_cut
            x_best = dict(x)
            if best_known:
                metrics.check_time_to_target(best_cut, best_known)

    elif T > 1e-10:
        # probabilistic acceptance of worsening moves
        prob = np.exp(delta / T)
        if rng.random() < prob:
            x[v] = 1 - x[v]
            gc.incremental_update(G, x, v)
            current_cut += delta
            non_improving_flips += 1
    else:

```

```

        non_improving_flips += 1

    # reheating
    if (non_improving_flips >= reheat_trigger and
        reheat_count < max_reheats):
        T = min(T * 10, T_initial * 0.5)
        beta = 1.0 / T
        remaining = 1.0 - elapsed_frac
        if remaining > 0.05:
            beta_initial = beta
            beta_final = beta * beta_final_multiplier
            non_improving_flips = 0
            reheat_count += 1

    metrics.record_cut(current_cut)

    return x_best, metrics

# -----
# Tabu Search
# -----

def tabu_search(G, budget_seconds, tabu_tenure=None,
                best_known=None, seed=None):
    """
    Tabu search for Max-Cut.
    Uses short-term memory to avoid cycling.
    """
    rng = np.random.default_rng(seed)
    metrics = Metrics()
    metrics.start()

    x = random_cut(G, rng)
    gc = GainCache()
    gc.update(G, x)

    x_best = dict(x)
    best_cut = compute_cut_value(G, x)
    current_cut = best_cut

    nodes = list(G.nodes())
    n = len(nodes)

    # scale tabu tenure with graph size if not specified
    if tabu_tenure is None:
        tabu_tenure = max(10, len(nodes) // 20)

```

```

# tabu list: tabu[v] = iteration when v becomes free again
tabu = {v: 0 for v in nodes}
iteration = 0

metrics.record_cut(current_cut)

while time.time() - metrics.start_time < budget_seconds:
    iteration += 1

    # find best non-tabu move (or aspiration)
    best_v = None
    best_gain = float('-inf')

    for v in nodes:
        gain = gc.gain[v]
        is_tabu = tabu[v] > iteration

        # aspiration: accept tabu move if it improves best known
        if is_tabu and (current_cut + gain) <= best_cut:
            continue

        if gain > best_gain:
            best_gain = gain
            best_v = v

    if best_v is None:
        # all moves tabu - pick least recently tabu
        best_v = min(nodes, key=lambda v: tabu[v])
        best_gain = gc.gain[best_v]

    # apply move
    x[best_v] = 1 - x[best_v]
    current_cut += best_gain
    gc.incremental_update(G, x, best_v)

    # update tabu list
    tabu[best_v] = iteration + tabu_tenure

    # update best
    if current_cut > best_cut:
        best_cut = current_cut
        x_best = dict(x)
        if best_known:
            metrics.check_time_to_target(best_cut, best_known)

    metrics.record_cut(current_cut)

```

```
return x_best, metrics
```

src/experiment.py

```
python
```

```
"""
```

```
Experiment runner – compares all methods across multiple trials.  
Produces the results needed for the thesis evaluation.
```

```
"""
```

```
import time  
import json  
import numpy as np  
from pathlib import Path  
from scipy import stats  
  
from src.graph import (load_gset, generate_random_regular,  
                        generate_erdos_renyi, generate_sbm, graph_stats)  
from src.gain_cache import GainCache  
from src.local_search import one_flip_ls, compute_cut_value, random_cut  
from src.qls import qls  
from src.adaptive_qls import adaptive_qls  
from src.baselines import simulated_annealing, tabu_search  
from src.backends import backend_exact, backend_neal, get_backend  
from src.selectors import (select_random, select_frustrated,  
                           select_frustrated_connected,  
                           select_impact, select_meta_rule, get_selector)  
from src.metrics import Metrics  
  
def run_single_trial(method_fn, method_kwargs, seed):  
    """Run one trial of a method with a given seed."""  
    method_kwargs = dict(method_kwargs)  
    method_kwargs['seed'] = seed  
    x_best, metrics = method_fn(**method_kwargs)  
    return x_best, metrics  
  
def run_experiment(G, methods, n_trials=30, best_known=None,  
                  results_dir=None, experiment_name='experiment'):  
    """Run a full experiment comparing multiple methods."""  
    results = {}  
  
    for method in methods:  
        name = method['name']  
        fn = method['fn']  
        kwargs = method['kwargs']  
  
        print(f"\n{'='*50}")  
        print(f"Running: {name} ({n_trials} trials)")  
        print(f"{'='*50}")
```

```

trial_results = {
    'best_cut': [],
    'escape_rate': [],
    'qls_calls': [],
    'mean_delta': [],
    'time_to_target': [],
    'approx_ratio': [],
}

for trial in range(n_trials):
    seed = trial * 1000 + 42
    x_best, metrics = run_single_trial(fn, kwargs, seed)

    best_cut = metrics.best_cut
    escape_rate = metrics.escape_rate()
    approx_ratio = (best_cut / best_known
                    if best_known else None)

    trial_results['best_cut'].append(best_cut)
    trial_results['escape_rate'].append(escape_rate)
    trial_results['qls_calls'].append(metrics.qls_calls)
    trial_results['mean_delta'].append(metrics.mean_improvement())
    trial_results['time_to_target'].append(
        metrics.time_to_target or float('nan')
    )
    if approx_ratio:
        trial_results['approx_ratio'].append(approx_ratio)

    if (trial + 1) % 5 == 0:
        print(f" Trial {trial+1:2d}/{n_trials} - "
              f"cut={best_cut:.1f} "
              f"escape={escape_rate:.3f}")

    results[name] = trial_results
    _print_method_summary(name, trial_results, best_known)

_print_statistical_comparison(results)

if results_dir:
    _save_results(results, results_dir, experiment_name)

return results

```

```

def _print_method_summary(name, trial_results, best_known):
    """Print summary statistics for one method."""

```

```

cuts = trial_results['best_cut']
escape_rates = trial_results['escape_rate']

print(f"\n {name} summary:")
print(f"    Best cut    - median={np.median(cuts):.1f} "
      f"IQR=[{np.percentile(cuts,25):.1f}, "
      f"{np.percentile(cuts,75):.1f}] "
      f"max={np.max(cuts):.1f}")
print(f"    Escape rate- median={np.median(escape_rates):.4f}")
if best_known:
    ratios = [c / best_known for c in cuts]
    print(f"    Approx ratio- median={np.median(ratios):.4f}")

def _print_statistical_comparison(results):
    """Wilcoxon signed-rank test between all method pairs."""
    names = list(results.keys())
    if len(names) < 2:
        return

    print(f"\n{'='*50}")
    print("Statistical comparison (Wilcoxon signed-rank test)")
    print(f"{'='*50}")

    for i in range(len(names)):
        for j in range(i + 1, len(names)):
            a = results[names[i]]['best_cut']
            b = results[names[j]]['best_cut']

            if len(a) < 2 or len(b) < 2:
                continue

            try:
                stat, p = stats.wilcoxon(a, b)
                sig = "***significant**" if p < 0.05 else "not significant"
                print(f" {names[i]} vs {names[j]}: "
                      f"p={p:.4f} ({sig})")
            except Exception as e:
                print(f" {names[i]} vs {names[j]}: "
                      f"test failed ({e})")

def _save_results(results, results_dir, experiment_name):
    """Save results to JSON."""
    results_dir = Path(results_dir)
    results_dir.mkdir(exist_ok=True)

```

```

filepath = results_dir / f"{experiment_name}.json"

def convert(obj):
    if isinstance(obj, (np.integer,)):
        return int(obj)
    if isinstance(obj, (np.floating,)):
        return float(obj)
    if isinstance(obj, np.ndarray):
        return obj.tolist()
    return obj

serialisable = {}
for method, data in results.items():
    serialisable[method] = {
        k: [convert(v) for v in vals]
        for k, vals in data.items()
    }

with open(filepath, 'w') as f:
    json.dump(serialisable, f, indent=2)

print(f"\nResults saved to {filepath}")

def run_quick_comparison(graph_type='random_regular',
                        n=50, budget=10, n_trials=5):
    """Quick comparison for development – small graph, few trials."""
    print(f"\nQuick comparison: {graph_type}, n={n}, "
          f"budget={budget}s, trials={n_trials}")

    if graph_type == 'random_regular':
        G = generate_random_regular(n, 3, seed=0)
    elif graph_type == 'erdos_renyi':
        G = generate_erdos_renyi(n, 0.15, seed=0)
    else:
        G = generate_sbm([n//2, n//2], 0.1, 0.3, seed=0)

    print("\nGraph statistics:")
    graph_stats(G)

    methods = [
        {
            'name': 'SA',
            'fn': simulated_annealing,
            'kwargs': {'G': G, 'budget_seconds': budget}
        },
        {

```

```

        'name': 'Tabu',
        'fn': tabu_search,
        'kwargs': {'G': G, 'budget_seconds': budget}
    },
    {
        'name': 'QLS-Random',
        'fn': qls,
        'kwargs': {
            'G': G, 'budget_seconds': budget,
            'k': 8, 'backend': backend_neal, 'n_reads': 50
        }
    },
    {
        'name': 'AQLS-Frustrated',
        'fn': adaptive_qls,
        'kwargs': {
            'G': G, 'budget_seconds': budget,
            'selector': select_frustrated,
            'backend': backend_neal,
            'k_min': 5, 'k_max': 20, 'n_reads': 50
        }
    },
    {
        'name': 'AQLS-Impact',
        'fn': adaptive_qls,
        'kwargs': {
            'G': G, 'budget_seconds': budget,
            'selector': select_impact,
            'backend': backend_neal,
            'k_min': 5, 'k_max': 20, 'n_reads': 50
        }
    },
]

results = run_experiment(
    G=G,
    methods=methods,
    n_trials=n_trials,
    results_dir=Path('results'),
    experiment_name=f'quick_{graph_type}_n{n}'
)

return results

```

```

def run_gset_experiment(instance_name, budget=30, n_trials=10):
    """Run a full comparison on a G-set instance."""

```

```

from pathlib import Path

filepath = f'data/gset/{instance_name}.txt'
G = load_gset(filepath)

print(f"\nG-set experiment: {instance_name}")
print(f"Budget: {budget}s per trial, {n_trials} trials")
graph_stats(G)

best_known_cuts = {
    'G1': 11624, 'G11': 564, 'G14': 3064, 'G22': 13359
}
best_known = best_known_cuts.get(instance_name)
if best_known:
    print(f"Best known cut: {best_known}")

# scale k with graph size
n = G.number_of_nodes()
k_small = max(10, n // 80)
k_min = max(5, n // 160)
k_max = max(30, n // 20)

print(f"k settings: fixed={k_small}, "
      f"adaptive min={k_min}, max={k_max}")

methods = [
    {
        'name': 'SA',
        'fn': simulated_annealing,
        'kwargs': {'G': G, 'budget_seconds': budget,
                   'best_known': best_known}
    },
    {
        'name': 'Tabu',
        'fn': tabu_search,
        'kwargs': {'G': G, 'budget_seconds': budget,
                   'best_known': best_known}
    },
    {
        'name': 'QLS-Random',
        'fn': qls,
        'kwargs': {
            'G': G, 'budget_seconds': budget,
            'k': k_small, 'backend': backend_neal,
            'n_reads': 200, 'best_known': best_known
        }
    },
]

```

```

{
    'name': 'AQLS-Frustrated',
    'fn': adaptive_qls,
    'kwargs': {
        'G': G, 'budget_seconds': budget,
        'selector': select_frustrated,
        'backend': backend_neal,
        'k_min': k_min, 'k_max': k_max,
        'n_reads': 200, 'best_known': best_known,
        'acceptance': 'improvement'
    }
},
{
    'name': 'AQLS-Impact',
    'fn': adaptive_qls,
    'kwargs': {
        'G': G, 'budget_seconds': budget,
        'selector': select_impact,
        'backend': backend_neal,
        'k_min': k_min, 'k_max': k_max,
        'n_reads': 200, 'best_known': best_known,
        'acceptance': 'improvement'
    }
},
{
    'name': 'AQLS-FrustratedConn',
    'fn': adaptive_qls,
    'kwargs': {
        'G': G, 'budget_seconds': budget,
        'selector': select_frustrated_connected,
        'backend': backend_neal,
        'k_min': k_min, 'k_max': k_max,
        'n_reads': 200, 'best_known': best_known,
        'acceptance': 'improvement'
    }
},
{
    'name': 'AQLS-FrustratedConn-LA',
    'fn': adaptive_qls,
    'kwargs': {
        'G': G, 'budget_seconds': budget,
        'selector': select_frustrated_connected,
        'backend': backend_neal,
        'k_min': k_min, 'k_max': k_max,
        'n_reads': 200, 'best_known': best_known,
        'acceptance': 'lookahead'
    }
}

```

```

        },
    ]

    results = run_experiment(
        G=G,
        methods=methods,
        n_trials=n_trials,
        best_known=best_known,
        results_dir=Path('results'),
        experiment_name=f'gset_{instance_name}'
    )

    return results


def run_k_sweep(instance_name, budget=30, n_trials=10):
    """
    Sweep k values to find the optimal subproblem size for AQLS.
    Tests k = 10, 20, 40, 80, 160 on the connected frustrated selector.
    """
    from pathlib import Path

    filepath = f'data/gset/{instance_name}.txt'
    G = load_gset(filepath)

    print(f"\nk sweep experiment: {instance_name}")
    print(f"Budget: {budget}s per trial, {n_trials} trials")
    graph_stats(G)

    best_known_cuts = {
        'G1': 11624, 'G11': 564, 'G14': 3064, 'G22': 13359
    }
    best_known = best_known_cuts.get(instance_name)
    if best_known:
        print(f"Best known cut: {best_known}")

    k_values = [10, 20, 40, 80, 160]
    methods = []

    methods.append({
        'name': 'SA',
        'fn': simulated_annealing,
        'kwargs': {'G': G, 'budget_seconds': budget,
                   'best_known': best_known}
    })

    methods.append({

```

```

        'name': 'QLS-Random-k10',
        'fn': qls,
        'kwargs': {
            'G': G, 'budget_seconds': budget,
            'k': 10, 'backend': backend_neal,
            'n_reads': 200, 'best_known': best_known
        }
    })

    for k in k_values:
        if k >= G.number_of_nodes():
            continue
        methods.append({
            'name': f'AQLS-FConn-LA-k{k}',
            'fn': adaptive_qls,
            'kwargs': {
                'G': G, 'budget_seconds': budget,
                'selector': select_frustrated_connected,
                'backend': backend_neal,
                'k_min': k, 'k_max': k,
                'n_reads': 200, 'best_known': best_known,
                'acceptance': 'lookahead'
            }
        })

    results = run_experiment(
        G=G,
        methods=methods,
        n_trials=n_trials,
        best_known=best_known,
        results_dir=Path('results'),
        experiment_name=f'k_sweep_{instance_name}'
    )

    return results

if __name__ == '__main__':
    run_quick_comparison(
        graph_type='random_regular',
        n=50,
        budget=10,
        n_trials=5
    )

```

python

```
"""
```

```
Unit tests for QUBO builder.
```

```
Critical test: K4 with unit weights has known optimal cut = 4.
```

```
"""
```

```
import pytest
import networkx as nx
from src.qubo import build_local_qubo, qubo_energy, merge_proposal
from src.gain_cache import GainCache
from src.local_search import compute_cut_value
```

```
def make_k4():
    """Complete graph on 4 vertices with unit weights."""
    G = nx.complete_graph(4)
    return G
```

```
def exact_solve(Q, S):
    """Brute-force exact solver for small subproblems."""
    S = list(S)
    n = len(S)
    best_energy = float('inf')
    best_x = None
    for mask in range(2 ** n):
        x_local = {}
        for i, v in enumerate(S):
            x_local[v] = (mask >> i) & 1
        e = qubo_energy(Q, x_local, S)
        if e < best_energy:
            best_energy = e
            best_x = dict(x_local)
    return best_x, best_energy
```

```
def test_k4_full_subproblem():
    """
    K4 with all 4 vertices in S and a fixed initial assignment.
    The optimal Max-Cut for K4 is 4 (bipartition {0,1} vs {2,3}).
    """
    G = make_k4()
    x = {0: 0, 1: 0, 2: 0, 3: 0}
    S = list(G.nodes())

    Q = build_local_qubo(G, x, S)
    x_local, _ = exact_solve(Q, S)
```

```

x_prop = merge_proposal(x, x_local, S)

cut = compute_cut_value(G, x_prop)
assert cut == 4.0, f"Expected cut=4 for K4, got {cut}"

def test_k4_partial_subproblem():
    """K4 with only 2 vertices in S, 2 fixed. Check boundary conditions."""
    G = make_k4()
    x = {0: 0, 1: 0, 2: 1, 3: 1}
    S = [0, 1]

    Q = build_local_qubo(G, x, S)
    x_local, _ = exact_solve(Q, S)
    x_prop = merge_proposal(x, x_local, S)

    cut = compute_cut_value(G, x_prop)
    assert cut >= 4.0, f"Expected cut >= 4 for K4 partial, got {cut}"

def test_qubo_energy_zero_assignment():
    """All zeros assignment should give zero energy."""
    G = make_k4()
    x = {0: 0, 1: 0, 2: 0, 3: 0}
    S = list(G.nodes())
    Q = build_local_qubo(G, x, S)
    x_local = {v: 0 for v in S}
    energy = qubo_energy(Q, x_local, S)
    assert energy == 0.0

def test_merge_proposal():
    """Merge should only update vertices in S."""
    x = {0: 0, 1: 0, 2: 0, 3: 0}
    x_local = {0: 1, 1: 1}
    S = [0, 1]
    x_prop = merge_proposal(x, x_local, S)
    assert x_prop[0] == 1
    assert x_prop[1] == 1
    assert x_prop[2] == 0
    assert x_prop[3] == 0

```

python

```

"""
Tests for graph loading, local search, selectors, baselines, and Adaptive QLS.
"""

import networkx as nx
import numpy as np
from src.graph import generate_random_regular, generate_erdos_renyi, generate_sbm
from src.gain_cache import GainCache
from src.local_search import one_flip_ls, compute_cut_value, random_cut

def test_random_regular_generation():
    G = generate_random_regular(10, 3, seed=42)
    assert G.number_of_nodes() == 10
    for v, d in G.degree():
        assert d == 3

def test_erdos_renyi_generation():
    G = generate_erdos_renyi(20, 0.3, seed=42)
    assert G.number_of_nodes() == 20

def test_one_flip_ls_reaches_local_optimum():
    G = generate_random_regular(20, 3, seed=42)
    rng = np.random.default_rng(42)
    x = random_cut(G, rng)
    gc = GainCache()

    x_opt, gc, at_opt, n_flips = one_flip_ls(G, x, gc)

    assert at_opt is True
    gc.assert_valid()
    for v in G.nodes():
        assert gc.gain[v] <= 0

def test_cut_improves_after_ls():
    G = generate_erdos_renyi(30, 0.3, seed=99)
    rng = np.random.default_rng(99)
    x = random_cut(G, rng)
    gc = GainCache()

    cut_before = compute_cut_value(G, x)
    x_opt, gc, at_opt, n_flips = one_flip_ls(G, x, gc)
    cut_after = compute_cut_value(G, x_opt)

```

```

    assert cut_after >= cut_before

def test_random_cut_valid():
    G = generate_random_regular(10, 3, seed=0)
    rng = np.random.default_rng(0)
    x = random_cut(G, rng)
    for v in G.nodes():
        assert x[v] in (0, 1)

def test_qls_runs_end_to_end():
    """QLS should run and produce a valid cut."""
    from src.qls import qls
    from src.backends import backend_exact

    G = generate_random_regular(20, 3, seed=42)
    x_best, metrics = qls(G, budget_seconds=5, k=6,
                          backend=backend_exact, seed=42)

    for v in G.nodes():
        assert x_best[v] in (0, 1)
    assert metrics.best_cut > 0
    print(f"\nQLS smoke test – best cut: {metrics.best_cut}")
    metrics.summary()

def test_sa_runs_end_to_end():
    from src.baselines import simulated_annealing

    G = generate_random_regular(20, 3, seed=42)
    x_best, metrics = simulated_annealing(G, budget_seconds=3, seed=42)

    for v in G.nodes():
        assert x_best[v] in (0, 1)
    assert metrics.best_cut > 0
    print(f"\nSA best cut: {metrics.best_cut}")

def test_tabu_runs_end_to_end():
    from src.baselines import tabu_search

    G = generate_random_regular(20, 3, seed=42)
    x_best, metrics = tabu_search(G, budget_seconds=3, seed=42)

    for v in G.nodes():

```

```

        assert x_best[v] in (0, 1)
    assert metrics.best_cut > 0
    print(f"\nTabu best cut: {metrics.best_cut}")

def test_all_selectors_return_valid_subsets():
    """All selectors should return valid vertex subsets of size <= k."""
    from src.selectors import (select_random, select_frustrated,
                               select_impact, select_meta_rule)

    G = generate_random_regular(20, 3, seed=42)
    rng = np.random.default_rng(42)
    x = random_cut(G, rng)
    gc = GainCache()
    gc.update(G, x)
    nodes = set(G.nodes())
    k = 6

    for sel_fn in [select_random, select_frustrated,
                   select_impact, select_meta_rule]:
        S = sel_fn(G, gc, k, rng=rng)
        assert len(S) <= k
        assert all(v in nodes for v in S)
        print(f"{sel_fn.__name__}: S={S}")

def test_frustrated_vs_impact_different():
    """Frustrated and impact selectors should pick different vertices."""
    from src.selectors import select_frustrated, select_impact

    G = generate_random_regular(30, 3, seed=42)
    rng = np.random.default_rng(42)
    x = random_cut(G, rng)
    gc = GainCache()

    x, gc, at_opt, _ = one_flip_ls(G, x, gc)

    k = 8
    S_frustrated = set(select_frustrated(G, gc, k))
    S_impact = set(select_impact(G, gc, k))

    assert S_frustrated != S_impact
    print(f"\nFrustrated: {sorted(S_frustrated)}")
    print(f"Impact:      {sorted(S_impact)}")

def test_adaptive_qls_all_selectors():

```

```

"""Adaptive QLS should run with all four selectors."""
from src.adaptive_qls import adaptive_qls
from src.backends import backend_exact
from src.selectors import (select_random, select_frustrated,
                           select_impact, select_meta_rule)

G = generate_random_regular(20, 3, seed=42)

selectors = {
    'random': select_random,
    'frustrated': select_frustrated,
    'impact': select_impact,
    'meta_rule': select_meta_rule,
}

results = {}
for name, sel in selectors.items():
    x_best, metrics = adaptive_qls(
        G=G,
        budget_seconds=5,
        selector=sel,
        backend=backend_exact,
        k_min=4,
        k_max=10,
        n_reads=None,
        seed=42
    )
    for v in G.nodes():
        assert x_best[v] in (0, 1)
    assert metrics.best_cut > 0
    results[name] = metrics.best_cut
    print(f"\nAdaptive QLS [{name}]: "
          f"cut={metrics.best_cut} "
          f"escape_rate={metrics.escape_rate():.3f} "
          f"calls={metrics.qls_calls}")

print(f"\nResults: {results}")

def test_frustrated_connected_is_connected():
    """
    Connected frustrated selector should return a connected subgraph
    on a connected graph.
    """
    from src.selectors import select_frustrated_connected

    G = generate_random_regular(30, 3, seed=42)

```

```

rng = np.random.default_rng(42)
x = random_cut(G, rng)
gc = GainCache()
x, gc, _, _ = one_flip_ls(G, x, gc)

k = 8
S = select_frustrated_connected(G, gc, k)

assert len(S) == k

subgraph = G.subgraph(S)
assert nx.is_connected(subgraph), \
    f"Subgraph is not connected: components = {list(nx.connected_components(subg

gains = sorted([abs(gc.gain[v]) for v in S])
print(f"\nConnected frustrated S gains: {[f'{g:.3f}' for g in gains]}")

def test_connected_vs_plain_frustrated_internal_edges():
    """
    Connected selector should have more internal edges than plain frustrated.
    Empirically: 11 vs 4 on test graph (2.75x more).
    """
    from src.selectors import select_frustrated, select_frustrated_connected

    G = generate_random_regular(30, 3, seed=42)
    rng = np.random.default_rng(42)
    x = random_cut(G, rng)
    gc = GainCache()
    x, gc, _, _ = one_flip_ls(G, x, gc)

    k = 8
    S_plain = select_frustrated(G, gc, k)
    S_conn = select_frustrated_connected(G, gc, k)

    def count_internal_edges(G, S):
        S_set = set(S)
        return sum(1 for u, v in G.edges()
                    if u in S_set and v in S_set)

    plain_internal = count_internal_edges(G, S_plain)
    conn_internal = count_internal_edges(G, S_conn)

    print(f"\nPlain frustrated internal edges: {plain_internal}")
    print(f"Connected frustrated internal edges: {conn_internal}")

```

```
# connected should have at least as many internal edges
assert conn_internal >= plain_internal
```

How to run

```
bash

# Activate venv
venv\Scripts\activate

# Run all tests (should show 17 passing)
python -m pytest tests/ -v

# Quick smoke test (5 methods, 50-vertex graph, 5 trials, 10s each)
python -m src.experiment

# Full G-set experiment (7 methods, 10 trials, 30s each)
python -c "from src.experiment import run_gset_experiment; run_gset_experiment('G11'

# k-sweep experiment (7 methods: SA, QLS-Random, AQLS-FConn-LA at k=10,20,40,80,160)
python -c "from src.experiment import run_k_sweep; run_k_sweep('G11', budget=30, n_t

# Other G-set instances
python -c "from src.experiment import run_gset_experiment; run_gset_experiment('G1',
```

Notes on what's NOT yet implemented

These are designed but not coded; they're listed in the project map (handoff doc §3) as future work:

- **Backends Phase 2:** SBM (OpenJij), GPU parallel SA (PyTorch), QAOA (PennyLane) — none have been written, only specified
- **Backends Phase 3:** GNN solver, LLM solver, Neuromorphic
- **Cascade architectures:** sequential cascade, warm-chain relay, parallel portfolio
- **Meta-selectors Level 2/3:** online logistic regression, contextual bandits
- **Breakout Local Search** (Benlic & Hao 2013) as a comparator baseline
- **Fiedler-augmented selector** (recommended by spectral research)
- **Belief propagation selector** (recommended by info-theory research)

The current code is the **Phase 1 implementation** — the minimum viable project that's already publishable.

Critical implementation notes (sign conventions, traps)

1. **QUBO sign:** `build_local_qubo` builds Q such that **minimising** Q maximises the Max-Cut contribution of S. K4 unit test verifies this.
2. **GainCache invalidation:** every accepted QLS proposal calls `gc.invalidate()`. Every selector calls `gc.assert_valid()` first. Stale gains hard-crash, never silently corrupt results.
3. **Zero-bias QUBO skip:** if Q is empty or all values near 0 (trivial subproblem), the iteration is skipped — prevents the solver from being called with no signal.
4. **Plateau detection:** when `frac_zero > 0.8` (over 80% of vertices have $|\text{gain}| < 1\text{e-}6$), selectors switch to degree-tiebreaker mode (`_plateau_aware_rank`).
5. **Pool guards:** clustering selector falls back to frustrated when `len(pool) < 5`.
6. **Tabu tenure** auto-scales: `max(10, len(nodes) // 20)`.
7. **neal import:** the package is installed via `pip install dwave-neal` but imported from `dwave.samplers` in modern versions.